

```
import UIKit
/*:
## App Exercise – Heart Rate Delegate
```

>These exercises reinforce Swift concepts in the context of a fitness tracking app.

Your fitness tracking app will likely have a class dedicated to receiving information from the fitness tracking hardware. The `HeartRateReceiver` class below represents a very simplified example of what this may look like with monitoring heart rate. The function `startHeartRateMonitoringExample` will generate random heart rates and assign them to `currentHR`, simulating how an instance of `HeartRateReceiver` may pick up on new heart rate readings at specific intervals.

`HeartRateViewController` below is a view controller that will present the heart rate information to the user. Throughout the exercises below you'll use the delegate pattern to pass information from an instance of `HeartRateReceiver` to the view controller so that anytime new information is obtained it is presented to the user.

```
*/
```

```
protocol HeartRateReceiverDelegate {
    func heartRateUpdated( to bpm: Int) -> String
}

class HeartRateReceiver {
    var currentHR: Int? {
        didSet {
            if let currentHR = currentHR {
                print("The most recent heart rate reading is \(currentHR).")
            } else {
                print("Looks like we can't pick up a heart rate.")
            }
        }
    }

    func startHeartRateMonitoringExample() {
        for _ in 1...10 {
            let randomHR = 60 + Int(arc4random_uniform(UInt32(15)))
            currentHR = randomHR
            Thread.sleep(forTimeInterval: 2)
        }
    }
}

class HeartRateViewController: UIViewController, HeartRateReceiverDelegate {
    var heartRateLabel: UILabel = UILabel()

    func heartRateUpdated( to bpm: Int) -> String {
        heartRateLabel.text = String(bpm)
    }
}
```

```

        return heartRateLabel.text!
    }
}
//: First, create an instance of `HeartRateReceiver` and call
`startHeartRateMonitoringExample`. Notice that every two seconds `currentHR`
get set and prints the new heart rate reading to the console.
let hr1 = HeartRateReceiver()
hr1.startHeartRateMonitoringExample()

```

/*:

In a real app, printing to the console does not show information to the user. You need a way of passing information from the `HeartRateReceiver` to the `HeartRateViewController`. To do this, create a protocol called `HeartRateReceiverDelegate` that requires a method `heartRateUpdated(to bpm:)` where `bpm` is of type `Int` and represents the new rate as `_beats per minute_`. Since playgrounds read from top to bottom and the two previously declared classes will need to use this protocol, you'll need to declare this protocol above the declaration of `HeartRateReceiver`.

Now make `HeartRateViewController` adopt the protocol you've just created. Inside the body of the required method you should set the text of `heartRateLabel` and print "The user has been shown a heart rate of <INSERT HEART RATE HERE>."

Now add a property called `delegate` to `HeartRateReceiver` that is of type `HeartRateReceiverDelegate?`. In the `didSet` of `currentHR` where `currentHR` is successfully unwrapped, call `heartRateUpdated(to bpm:)` on the `delegate` property.

Finally, return to the line of code just after you initialized an instance of `HeartRateReceiver`. Initialize an instance of `HeartRateViewController`. Then, set the `delegate` property of your instance of `HeartRateReceiver` to be the instance of `HeartRateViewController` that you just created. Wait for your code to compile and observe what is printed to the console. Every time that `currentHR` gets set, you should see both a printout of the most recent heart rate, and the print statement stating that the heart rate was shown to the user.

Copyright © 2021 Apple Inc.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

[Previous](@previous) | page 5 of 5
*/